

!!!!!!!!!!!!!!!!!!!! IMPORTANT !!!!!!!!!!!!!!!!!!!!!

- Do not use a built-in equivalent to implement your solutions. E.g., DON'T USE *last* in the body of *mylast*. This also applies, if a built-in procedure exists but has a different name.
- We assume that the lists contain only numbers (integers). Also we assume them to be flat, i.e., they contain no further lists.

1.

a) Implement procedure (**mylast xs**). It shall return the last element in the given list **xs**.

E.g. (mylast (list 1 2 3)) → 3

b) Implement procedure (**tailLength xs**) that returns the length of a list. It shall be a tail-recursive version of (my)length-procedure discussed in the lecture.

2. Write a recursive procedure (**getFromIdx idx xs**) that returns a list with the elements of list **xs** starting from index **idx**.

E.g. (getFromIdx 2 (list 0 1 2 3 4 5 6)) → (list 2 3 4 5 6).

3. Write a function (**combine op xs ys**) that takes a function **op** and two lists **xs** and **ys** as input. It returns a new list by applying **op** on the elements with matching index. We can assume **|xs| = |ys|**.

E.g. (combine + (list 1 2 3) (list 4 5 6)) → '(5 7 9)

4. Write a recursive procedure (**isSame xs ys**) that returns **#t** iff (if and only if) the two lists **xs** and **ys** are the same (they contain exactly the same elements in the same order).

E.g. (isSame (list 1 2) (list 1)) → #f

E.g. (isSame (list 1 2) (list 2 1)) → #f

E.g. (isSame (list 1 2) (list 1 2)) → #t

5. Take a look at the procedure **minbad**. It shall return the minimum value of a list **xs**, but it has the reputation of performing very inefficiently.

```
(define (minbad xs)
  (cond [(null? (cdr xs)) (car xs)]
        [(< (car xs) (minbad (cdr xs))) (car xs) ]
        [else (minbad (cdr xs))]))
```

- a) show that it performs pretty bad in the average case.
- b) explain why this is the case
- c) implement a more efficient version